BUD DETECTOR - REPORT

Introduction-

We trained a YOLOv8 object detection model on a dataset of 100 annotated images. The model was designed to locate and draw a bounding box around every visible bud in an image, enabling an inference script to process entire directories of photos automatically.
The main challenge was the small dataset size (100 images) combined with a dense annotation task (100–230 buds per image). Despite these constraints, the model successfully learned to detect buds across a variety of field conditions.

Dataset handling-

I used random.seed(42) to set a fixed random state before shuffling the dataset with random.shuffle(). This guarantees that the exact data partitioning is fully reproducible by another engineer. After ensuring a balanced distribution, I split the dataset into an 80% training set (80 images) and 20% validation set (20 images).

The dataset's original annotations were provided in a four-point polygon format. Because the YOLO architecture requires a standard bounding box format, I wrote a custom preprocessing script to calculate the center the dimensions of the buds from the polygon points.

While it is standard practice to divide into training, validation and test sets (typically using 80/10/10 split), a separate test set was omitted due to the extreme data scarcity. Holding out additional images for testing would have further reduced the already small training set increasing the risk of overfitting and unstable performance estimates. Instead, model performance was monitored using a validation split together with augmentation and early stopping to reduce overfitting.

Model choice-

YOLOv8n.
I chose it because the dataset is very small, containing only 100 labeled images.
With such limited data, larger YOLO models may easily overfit and memorize the training images instead of learning patterns that generalize. YOLOv8n is smaller and lighter which helps reduce the risk of overfitting. In addition, it is faster and requires less computational power, making it suitable for real time inference in practical application.

Preprocessing and augmentation-

For the selected baseline model the default ultralytics YOLOv8 preprocessing and augmentation pipeline was utilized. This approach provided a solid foundation for the model to generalize well, without heavily distorting the natural shape of the small buds.

Preprocessing:

- **Resizing:** All input images were resized to a standard resolution of 640X640. To maintain the original aspect ratio of the buds, automatic padding was applied to the edges.
- **Normalization:** Images pixel values were automatically scaled to stabilize the training process.

Augmentation:

- **Moasic (1.0):** This technique merges four different training images into a single frame. It forces the model to learn context and improves its ability to detect very small buds.
- **Horizontal flip (0.5):** Images are flipped horizontally with a 50% probability. This naturally increases the variety of bud orientations in the data.
- **Spatial scalling (0.5) and Translation(0.1):** Applied random zooming and slight shifting. This teaches the model to recognize buds regardless of the camera's distance or the bud's position in the frame.
- **Color Jitter (HSV adjustments):** Randomly alters the image colors, brightness and contrast. This is highly effective for agriculture data as it simulates real outdoor conditions like shifting sunlight and shadows on the buds.

Training setup-

- **Hardware and Weights:**
  The model was initialized with COCO pretrained weights for transfer learning. Due to hardware limitations the training was executed entirely on a CPU.
- **Batch size:**
  I used a batch size of 8. This accommodated the CPU memory constraints and introduced slight gradient noise during training, which helps the network generalize better to unseen data.
- **Optimizer and Learning Rate**:
  The training utilized the default YOLOv8 SGD optimizer momentum=0.937 with an initial learning rate of 0.01 and a 3 epoch warmup. While I tested other optimizers like AdamW in separate runs, the default SGD setup proved to be the most stable and provided the highest F1 score for this dataset.
- **Epochs and Early Stopping**:
  To implement the early stopping strategy mentioned earlier, I defined a maximum of 100 epochs with a patience of 20. The training stopped automatically at epoch 82 when the validation metrics ceased to improve, and the best weights were successfully saved from epoch 62.

Validation strategy-

To monitor the model's performance I used the 20 images from the 80/20 split as a fixed validation set. This set was used exclusively to evaluate the model during training.

- **Process**: Validation was performed after every epoch. The model automatically saved the best.pt weights based on the best mAP@50 score achieved, ensuring the most robust version of the model was retained.
- **Limitations:** It is important to note that with only 20 validation images, the metrics have high variance. A single difficult or easy image can shift the mAP by several percentage points, which should be considered when interpreting the results.

Metrics and qualitative results-

Multiple experiments were conducted, varying optimizers (AdamW vs. default), learning rates and extreme augmentations.

```
results = model.train(
    data='data.yaml',
    epochs=100,
    patience=20,
    batch=8,
    imgsz=640,
    optimizer='AdamW',
    lr0=0.001,
    seed=42,
    deterministic=True,
    mosaic=1.0,
    hsv_h=0.015,
    hsv_s=0.7,
    hsv_v=0.4,
    degrees=10.0,
    fliplr=0.5,
    project='bud_detector_with_adamw',
    name='yolov8n_custom'
)
```

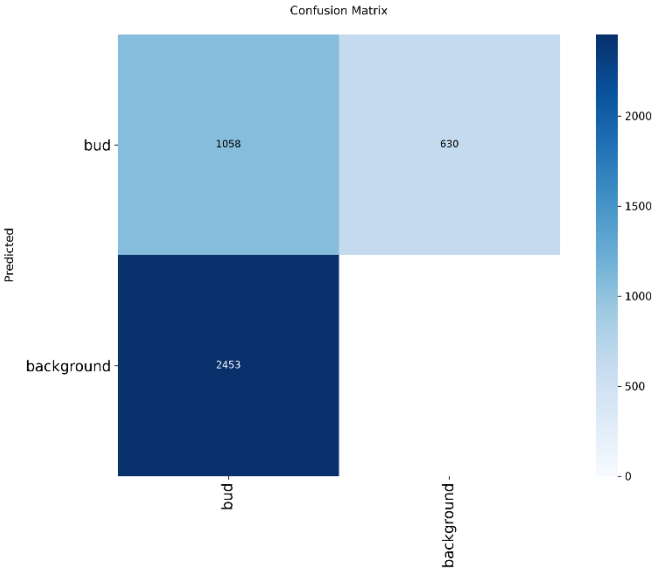
Ultimately the baseline YOLOv8 configuration yielded the most stable performance for this specific dataset size.

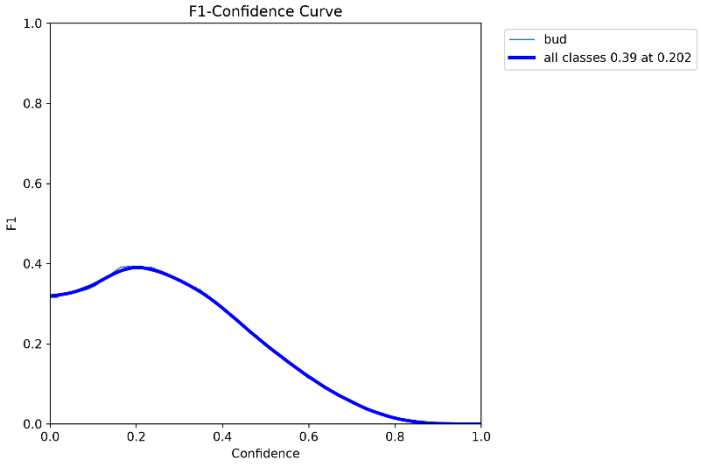
Peak F1 score: 0.39 at a confidence threshold of 0.202.
TP:1058 buds correctly identified within the validation set.
mAP@0.5: 0.301. This represents the overall accuracy of the model in detecting buds. Given that we only trained on 80 images, this score confirms that the model has successfully identified the core visual patterns of the buds.

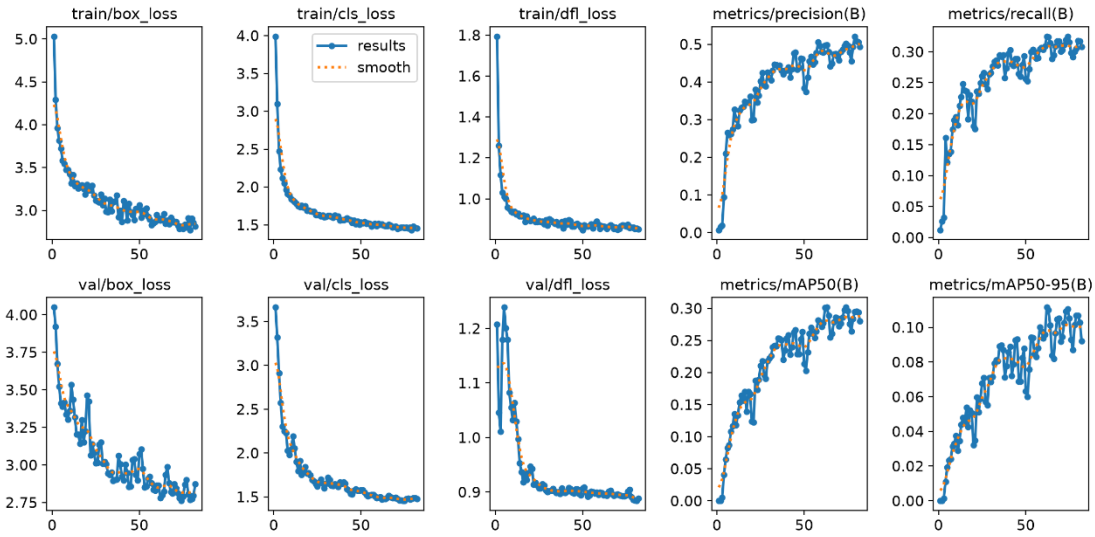
Performance Analysis:

- The training and validation loss curves show that the model learned the visual patterns of the buds effectively.
- The Precision-Confidence curve shows the model is highly reliable at higher thresholds (Precision reaches 1.00 at 0.804 confidence).
- The lower recall indicates that the model is somewhat cautious, missing buds that are obscured or have low contrast against the background.

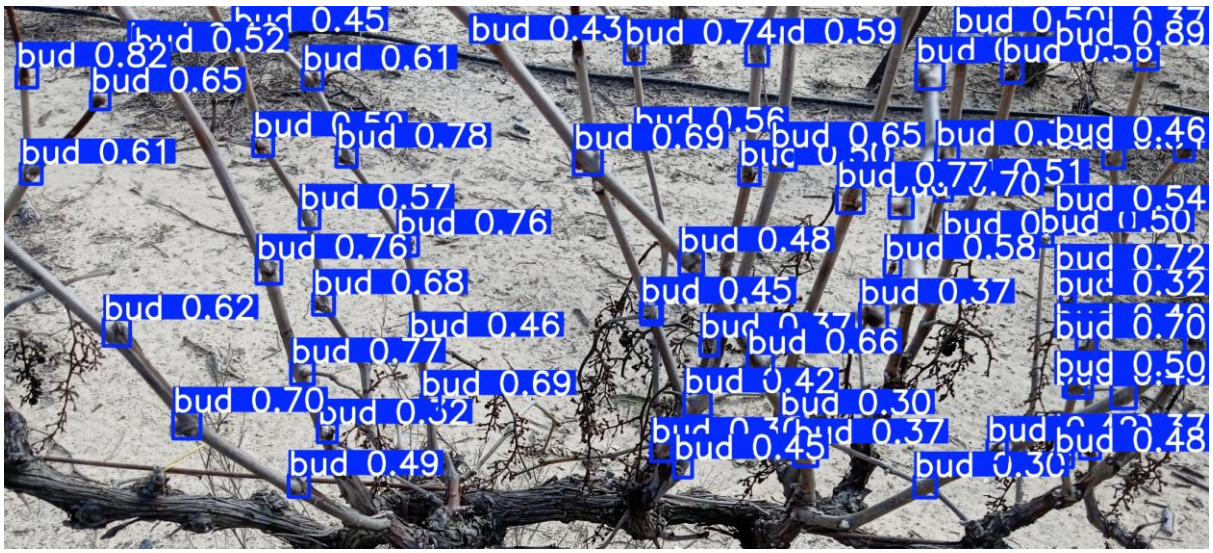
Training Outcome: Training stopped at epoch 82 via early stopping to prevent further overfitting, with the best performance captured at epoch 62.







Successful example:



Failure cases and next steps-

Analysis of the final model's confusion matrix reveal the primary challenges and failure modes:

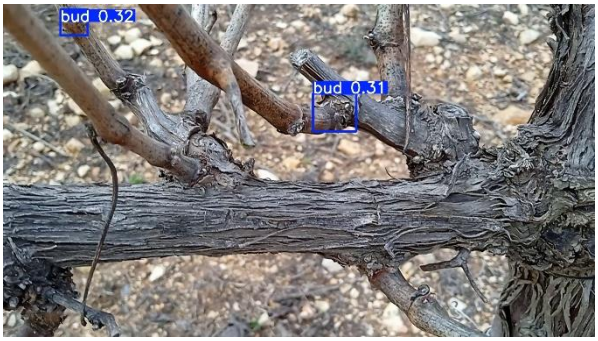
Primary Failure Modes:

- **False Negatives:** The model struggles with buds that are heavily occluded by foliage, obscured by deep shadows, or extremely small relative to the image size.
- **False Positives:** Certain background elements specifically leaf textures, stems or soil patterns that mimic the bud's geometry are occasionally misclassified.

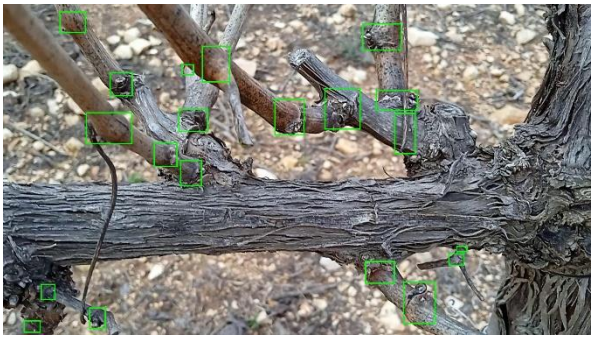
Next Steps:

- **Data Expansion:** Increasing the dataset size to 1,000 and more images is the most effective way to improve robustness and reduce overfitting.
- **Hard Negative Mining:** Including background only images in the training set (images with no buds) will significantly help the model learn to ignore irrelevant agricultural textures.

Example of bad detection:



What we expected to get:



Reproducibility-

1. Make sure you are working inside the directory which includes the dataset.

2. ***Install:***
   pip install ultralytics

3. ***Prepare dataset***:
   python prepare_dataset.py

4. ***Train:***
   Python train.py

5. ***Inference over a directory:***
   Python inference.py –source path/to/images –weights runs/detect/runs/bud_detector/weights/best.pt